

Komite - Diseño funcional del producto

1. Visión general

Komite es una plataforma para mantener informado al comité de administración, a los vecinos y al equipo interno de la administradora sobre todo lo que ocurre en los condominios.

No reemplaza sistemas administrativos como Edifito. Komite se enfoca en la operación diaria, la trazabilidad, la comunicación y la generación automática de informes.

Komite debe diseñarse desde el inicio como una plataforma **multi-tenant SaaS**. En este contexto, un **tenant** equivale a una empresa administradora o empresa de gestión de condominios que contrata Komite.

Ejemplo:

```
Vallestelar vende Komite
    ↓
Empresa Administradora A contrata Komite = Tenant A
Empresa Administradora B contrata Komite = Tenant B
    ↓
Cada tenant gestiona sus propios condominios, empleados, comités y vecinos
```

Regla base:

```
Vallestelar administra la plataforma SaaS.
Cada empresa compradora administra sus propios condominios dentro de Komite.
Los datos de un tenant nunca deben mezclarse ni ser visibles para otro tenant.
```

La idea central es:

```
Empleado / Conserje / Supervisor / Administrador
    ↓
Carga información en Komite
    ↓
Komite organiza, clasifica y redacta con IA
    ↓
Comité y vecinos reciben información clara y oportuna
```

2. Problema que resuelve

En muchos condominios existe una brecha entre lo que la administración hace y lo que el comité o los vecinos perciben.

Problemas habituales:

- El comité no sabe en qué estado están las tareas.
- Los vecinos sienten que no se les informa.
- Los supervisores reportan por WhatsApp de forma desordenada.
- Las fotos, audios y mensajes quedan perdidos en chats.
- Los informes se redactan manualmente.
- No existe historial claro de incidencias, mantenciones y compromisos.

Komite busca convertir la operación diaria en información estructurada, verificable y comunicable.

3. Propuesta de valor

Komite transforma mensajes, audios, fotos y tareas internas en informes, comunicaciones y trazabilidad para condominios.

Frases comerciales posibles:

- **Komite: todo lo que ocurre en tu condominio, informado a tiempo.**
 - **Komite: transparencia operativa para comités y comunidades.**
 - **Komite: de la gestión diaria al informe automático.**
-

4. Usuarios del sistema

4.1 Empleados del condominio

Ejemplos:

- Conserjes.
- Auxiliares.
- Personal de aseo.
- Mantención interna.

Uso principal:

- Reportar incidencias.
 - Subir fotos.
 - Registrar novedades.
 - Informar trabajos realizados.
-

4.2 Supervisores

Uso principal:

- Crear inspecciones.
- Validar incidencias.
- Completar checklists.
- Subir evidencias.

- Generar informes de terreno.
 - Marcar tareas como resueltas.
-

4.3 Administradores

Uso principal:

- Revisar la operación de todos los condominios.
 - Aprobar informes generados por IA.
 - Comunicar al comité o vecinos.
 - Asignar tareas.
 - Revisar indicadores.
 - Controlar pendientes.
-

4.4 Comité de administración

Uso principal:

- Ver tablero del condominio.
 - Revisar incidencias abiertas.
 - Ver informes mensuales.
 - Revisar mantenciones realizadas.
 - Consultar historial.
 - Aprobar o comentar ciertas solicitudes.
-

4.5 Vecinos

Uso principal:

- Recibir comunicados.
 - Consultar novedades del condominio.
 - Reportar incidencias, si se habilita.
 - Ver estados generales publicados por la administración.
-

4.6 Empleados internos de la empresa administradora compradora de Komite

Son los usuarios de la empresa que contrata Komite como tenant.

Uso principal en la web operativa Nuxt:

- Gestionar condominios propios del tenant.
- Gestionar usuarios internos del tenant.
- Revisar incidencias, tareas e inspecciones.
- Preparar comunicaciones para comité y vecinos.
- Aprobar informes generados por IA.
- Controlar indicadores operativos de sus condominios.
- Revisar actividad de supervisores y empleados.

4.7 Empleados internos de Vallestelar

Vallestelar es la empresa que vende y opera Komite como plataforma SaaS.

Uso principal en el backoffice SaaS:

- Crear y administrar tenants.
 - Activar o desactivar empresas administradoras.
 - Configurar planes, límites y módulos disponibles.
 - Definir parámetros globales del sistema.
 - Revisar salud de la plataforma.
 - Gestionar configuración avanzada de IA, canales e integraciones.
 - Auditar incidencias técnicas o de soporte entre tenants sin mezclar información operativa innecesaria.
-

5. Plataformas necesarias

Komite tendrá cuatro interfaces principales.

5.1 App móvil y tablet en .NET MAUI

Dirigida a:

- Supervisores.
- Conserjes.
- Personal de terreno.
- Comité.
- Vecinos.

Uso recomendado:

- Móvil para reportes rápidos, fotos, audios y novedades.
 - Tablet para supervisores en terreno.
 - Tablet para reuniones de comité.
-

5.2 Web operativa del tenant en Nuxt

Dirigida a los empleados de la empresa administradora que compra Komite.

Ejemplos de usuarios:

- Administradores de condominios.
- Supervisores senior.
- Personal administrativo.
- Encargados de comunicaciones.
- Jefaturas de operación.

Responsabilidad:

- Operación diaria de los condominios del tenant.

- Gestión de incidencias, tareas, inspecciones e informes.
- Aprobación de borradores generados por IA.
- Comunicación con comité y vecinos.
- Dashboard operativo por empresa y por condominio.

Esta web será el producto principal para la empresa compradora.

5.3 Portal web para comité y vecinos

Puede resolverse inicialmente dentro de la app MAUI y/o mediante vistas web responsivas.

Dirigida a:

- Comité de administración.
- Vecinos.
- Propietarios.
- Residentes.

Funcionalidades:

- Iniciar sesión.
 - Ver comunicados.
 - Ver informes publicados.
 - Revisar estado del condominio.
 - Consultar historial visible.
 - Reportar incidencias si se habilita.
-

5.4 Backoffice SaaS de Vallestelar

Dirigido exclusivamente a Vallestelar como empresa que vende y opera Komite.

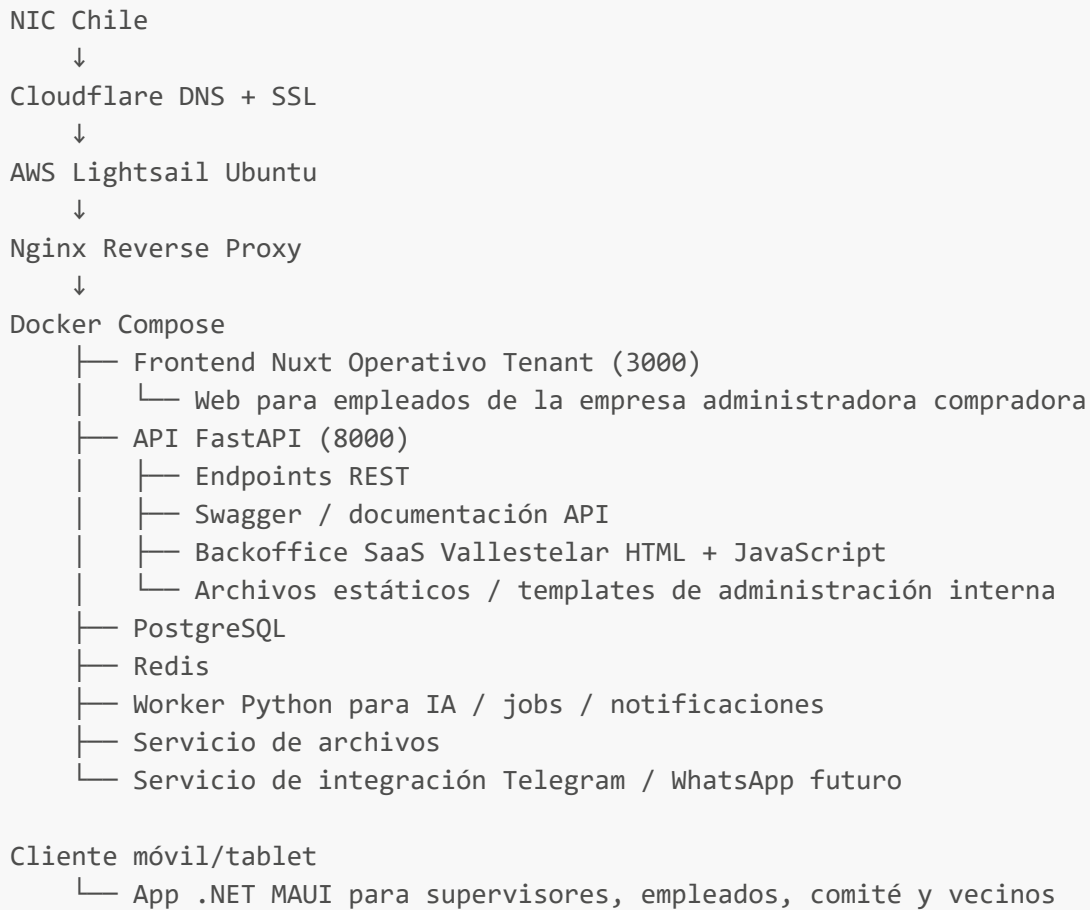
Responsabilidad:

- Configuración global del producto.
- Gestión de tenants.
- Planes y límites de uso.
- Configuración avanzada de módulos.
- Parámetros de IA.
- Canales de comunicación disponibles.
- Auditoría global.
- Soporte técnico.

Este backoffice no es el panel diario de las empresas administradoras. Es una herramienta interna de Vallestelar.

6. Arquitectura funcional propuesta

Komite mantendrá una arquitectura similar al proyecto base en Lightsail, Nginx, Docker Compose, PostgreSQL y Redis, pero adaptada a un SaaS multi-tenant y a las nuevas interfaces definidas.



6.1 Separación conceptual de interfaces

Web operativa Nuxt

Es la web que usará cada empresa administradora que compra Komite.

Ejemplo:

```

operacion.komite.cl
app.komite.cl
cliente.komite.cl

```

Objetivo:

- Gestión diaria del tenant.
- Condominios propios.
- Usuarios propios.
- Tareas, incidencias, inspecciones, informes y comunicaciones.

Backoffice SaaS Vallestelar

Es el panel interno de Vallestelar.

Ejemplo:

```
admin.komite.cl  
backoffice.komite.cl
```

Objetivo:

- Crear tenants.
- Configurar planes.
- Habilitar módulos.
- Revisar métricas globales.
- Configurar proveedores de IA.
- Revisar logs, auditoría y soporte.

Este backoffice puede mantenerse inicialmente en **HTML + JavaScript servido desde FastAPI**, porque es de uso interno y no requiere tanta experiencia visual como la web operativa del cliente.

App móvil/tablet .NET MAUI

Es la herramienta de terreno y de consulta rápida.

Objetivo inicial:

- Supervisores y empleados alimentan el sistema con fotos, audios, checklists e incidencias.

Objetivo posterior:

- Comité y vecinos consultan comunicados, informes y novedades.
-

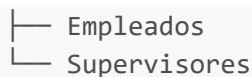
6.2 Modelo multi-tenant

Definición:

```
Tenant = empresa administradora o empresa de gestión de condominios que contrata  
Komite.
```

Jerarquía funcional:

```
Vallestarlar  
├── Tenant / Empresa administradora  
│   ├── Condominios  
│   │   ├── Torres / sectores  
│   │   ├── Unidades / departamentos  
│   │   ├── Comité  
│   │   └── Vecinos
```



Reglas:

- Un tenant puede tener muchos condominios.
- Un condominio pertenece a un solo tenant.
- Un usuario puede pertenecer a uno o varios condominios dentro del mismo tenant.
- Vallestelar puede ver y administrar tenants desde el backoffice SaaS.
- Un tenant no puede ver datos de otro tenant.
- Todas las tablas operativas deben tener `tenant_id` directa o indirectamente.

6.3 Decisión tecnológica actualizada

MVP / primera versión comercial

- └─ App móvil/tablet .NET MAUI
- └─ Web operativa Nuxt para empleados del tenant
- └─ API FastAPI
- └─ Backoffice SaaS Vallestelar HTML + JavaScript servido desde FastAPI
- └─ PostgreSQL
- └─ Redis
- └─ Worker Python para IA, informes y notificaciones
- └─ Integración inicial con Telegram para operación interna

Fase posterior

- └─ Portal web específico para vecinos/comité si la app MAUI no cubre bien esa experiencia
- └─ WhatsApp Business Cloud API si se valida el modelo de comunicación con residentes
- └─ Separación del backoffice SaaS en un frontend más robusto si Vallestelar lo necesita

Motivo:

- Nuxt aporta una mejor experiencia para el uso diario de empleados de la empresa compradora.
- FastAPI se mantiene como API central y como servidor del backoffice interno simple de Vallestelar.
- .NET MAUI se reserva para uso móvil/tablet con cámara, audio, archivos, notificaciones y trabajo en terreno.
- La arquitectura queda preparada para vender Komite a múltiples empresas administradoras.

7. Arquitectura por componentes

7.1 Backoffice SaaS Vallestelar

Tecnología definida para el MVP:

- FastAPI.
- HTML.

- JavaScript.
- CSS.
- Templates Jinja2 o archivos estáticos servidos desde FastAPI.
- Fetch/AJAX contra endpoints REST internos.

Responsabilidad:

- Panel interno de Vallestelar.
- Gestión de tenants.
- Activación y suspensión de empresas administradoras.
- Configuración de planes y límites.
- Configuración global de módulos.
- Configuración de proveedores de IA.
- Configuración de canales de comunicación.
- Revisión de logs, auditoría y soporte.

Enfoque:

Este backoffice debe ser funcional y seguro. No será la interfaz diaria de los clientes, por lo que puede mantenerse simple al inicio.

Pantallas iniciales:

- Login Vallestelar.
- Dashboard SaaS.
- Tenants / empresas administradoras.
- Planes y módulos.
- Configuración global de IA.
- Configuración de canales.
- Auditoría global.
- Logs técnicos.
- Soporte.

7.2 Web operativa Nuxt para tenants

Tecnología definida:

- Nuxt.
- Consumo de API REST FastAPI.
- Autenticación JWT.
- Control estricto por `tenant_id`, rol y condominio.

Responsabilidad:

- Portal diario para empleados de la empresa administradora que compra Komite.
- Gestión de condominios propios del tenant.
- Gestión de usuarios propios.
- Revisión de incidencias.
- Gestión de tareas.

- Inspecciones y checklists.
- Aprobación de informes generados por IA.
- Publicación de comunicados.
- Panel de métricas por empresa y por condominio.

Pantallas iniciales:

- Login tenant.
 - Dashboard empresa.
 - Dashboard condominio.
 - Condominios.
 - Incidencias.
 - Tareas.
 - Inspecciones.
 - Informes.
 - Comunicaciones.
 - Usuarios del tenant.
 - Comité y vecinos.
 - Configuración operativa del tenant.
-

7.3 App móvil / tablet en .NET MAUI

Tecnología definida:

- .NET MAUI.
- Consumo de API REST FastAPI.
- Almacenamiento local opcional para modo offline.
- Acceso a cámara, micrófono, galería y archivos.
- Notificaciones push en fase posterior.

Dirigida a:

- Supervisores.
- Conserjes.
- Personal de terreno.
- Comité.
- Vecinos.

Uso principal por perfil:

- Supervisores: inspecciones, fotos, audios, checklists, cierre de tareas.
- Conserjes/empleados: reportar novedades e incidencias.
- Comité: revisar estado del condominio, informes y pendientes.
- Vecinos: ver comunicados, reportar incidencias si se habilita, consultar información publicada.

Recomendación:

La app MAUI debe comenzar enfocada en supervisores y empleados, porque ellos son quienes alimentan el sistema. Después se puede ampliar al comité y vecinos.

7.4 API principal

Tecnología definida:

- FastAPI.
- Tortoise ORM + Aerich o SQLAlchemy + Alembic, preferentemente manteniendo el patrón del proyecto base si ya está validado.
- PostgreSQL como base de datos principal.
- Redis para colas, cache y estados temporales.

Responsabilidad:

- Autenticación.
 - Condominios.
 - Usuarios.
 - Roles y permisos.
 - Tareas.
 - Incidencias.
 - Informes.
 - Comunicaciones.
 - Archivos.
 - Integración con IA.
 - Integración con Telegram.
 - Preparación futura para WhatsApp Business Cloud API.
-

7.5 Base de datos

Tecnología:

- PostgreSQL.

Responsabilidad:

- Datos estructurados.
 - Usuarios.
 - Condominios.
 - Tareas.
 - Incidencias.
 - Informes.
 - Comunicaciones.
 - Auditoría.
-

7.6 Redis

Uso recomendado:

- Colas de procesamiento.
- Estados temporales.

- Jobs en segundo plano.
 - Cache de dashboards.
 - Procesamiento de IA.
 - Eventos de notificación.
-

7.7 Servicio de IA

Puede estar integrado al backend o separado como worker.

Responsabilidades:

- Transcribir audios.
- Clasificar mensajes.
- Resumir novedades.
- Generar informes.
- Redactar comunicados.
- Transformar lenguaje informal en texto profesional.

Modelos posibles:

- OpenAI.
- Mistral.
- Claude.
- Gemini.

Recomendación:

- Empezar con API externa.
 - No entrenar modelo propio al inicio.
 - Usar plantillas rígidas para controlar la salida.
 - Guardar siempre el texto original, la salida generada y el usuario que aprobó el resultado.
-

7.8 Servicio de archivos

Debe guardar:

- Fotos.
- Audios.
- PDFs.
- Documentos.
- Evidencias.

Opciones:

- Volumen local en Lightsail para MVP.
 - S3 o compatible S3 para producción más robusta.
-

7.9 Servicio de notificaciones

Canales:

- Telegram para operación interna inicial.
 - Correo electrónico.
 - Push móvil en fase posterior.
 - WhatsApp Business API en fase posterior.
-

7.10 Evolución futura de frontends

La web operativa Nuxt forma parte de la primera versión comercial para empleados de los tenants.

Lo que queda como evolución futura es:

- Portal Nuxt separado para comité y vecinos si se requiere una experiencia web más completa.
- Backoffice SaaS Vallestar en Nuxt si el panel interno crece demasiado.
- Apps o frontends específicos por marca blanca para tenants grandes.

Arquitectura posible a futuro:

```
Docker Compose
├── Frontend Nuxt Operativo Tenant (3000)
├── Frontend Nuxt Portal Comité/Vecinos (3001)
├── API FastAPI (8000)
├── PostgreSQL
├── Redis
└── Worker IA / notificaciones
```

8. Módulos funcionales

8.1 Módulo de tenants

Permite a Vallestar gestionar las empresas administradoras que compran Komite.

Campos principales:

- Nombre legal de la empresa.
- Nombre comercial.
- RUT.
- Dominio o subdominio asignado.
- Plan contratado.
- Estado del tenant: activo, suspendido, demo, cancelado.
- Límites: usuarios, condominios, almacenamiento, solicitudes IA, comunicaciones.
- Módulos habilitados.
- Contacto principal.
- Fecha de alta.

Funciones:

- Crear tenant.
 - Activar/suspender tenant.
 - Configurar plan.
 - Definir límites.
 - Habilitar módulos.
 - Ver métricas globales de uso.
 - Acceder a soporte técnico controlado.
-

8.2 Módulo de condominios

Permite gestionar cada comunidad dentro de un tenant.

Campos principales:

- Nombre del condominio.
- Dirección.
- Torres.
- Cantidad de unidades.
- Comité asociado.
- Administrador responsable.
- Supervisores asignados.
- Canales de comunicación habilitados.

Funciones:

- Crear condominio.
 - Editar datos.
 - Asignar usuarios.
 - Configurar reglas de comunicación.
 - Configurar categorías de incidencias.
-

8.3 Módulo de usuarios y roles

Roles iniciales:

- Superadmin Vallestar.
- Soporte Vallestar.
- Owner tenant / dueño empresa administradora.
- Administrador empresa.
- Administrador condominio.
- Supervisor.
- Conserje / empleado.
- Comité.
- Vecino.

Permisos por rol:

Rol	Crear incidencia	Aprobar informe	Ver comité	Ver vecinos	Gestionar usuarios
Superadmin	Sí	Sí	Sí	Sí	Sí
Administrador empresa	Sí	Sí	Sí	Sí	Sí
Administrador condominio	Sí	Sí	Sí	Sí	Parcial
Supervisor	Sí	No	Parcial	No	No
Conserje	Sí	No	No	No	No
Comité	No / opcional	No	Sí	Parcial	No
Vecino	Opcional	No	No	Sí	No

8.4 Módulo de incidencias

Una incidencia es cualquier hecho reportado en el condominio.

Ejemplos:

- Falla de ascensor.
- Fuga de agua.
- Problema eléctrico.
- Reclamo por ruido.
- Portón detenido.
- Sensor sin funcionamiento.

Campos:

- Condominio.
- Categoría.
- Prioridad.
- Estado.
- Descripción original.
- Descripción profesional generada por IA.
- Fotos.
- Audios.
- Responsable.
- Fecha de creación.
- Fecha compromiso.
- Fecha de cierre.

Estados:

Nueva
En revisión

Asignada
En proceso
Pendiente proveedor
Resuelta
Informada
Cerrada

8.5 Módulo de tareas

Las tareas son acciones concretas derivadas de incidencias, mantenciones o solicitudes del comité.

Ejemplo:

Tarea: Revisar bomba N°2
Responsable: Supervisor técnico
Fecha límite: 30/06/2026
Checklist:
☐ Revisar presión
☐ Revisar ruido
☐ Revisar tablero
☐ Adjuntar fotos

Funciones:

- Crear tarea manual.
- Crear tarea desde incidencia.
- Asignar responsable.
- Agregar checklist.
- Adjuntar evidencia.
- Cambiar estado.
- Generar informe al cerrar.

8.6 Módulo de inspecciones

Pensado para supervisores.

Tipos:

- Revisión de bombas.
- Revisión de ascensores.
- Revisión de iluminación.
- Revisión de piscina.
- Revisión de estacionamientos.
- Revisión de sala de basura.
- Revisión general semanal.

Cada inspección puede tener:

- Checklist.
- Fotos obligatorias.
- Observaciones.
- Firma o confirmación.
- Geolocalización opcional.
- Informe automático.

8.7 Módulo de informes

Tipos de informes:

- Informe de incidencia.
- Informe de mantención.
- Informe de inspección.
- Informe mensual de gestión.
- Informe para comité.
- Informe para vecinos.

Flujo:

```
graph TD; A[Supervisor carga información] --> B[IA genera borrador]; B --> C[Administrador revisa]; C --> D[Administrador aprueba]; D --> E[Komite publica o envía];
```

Estructura de informe:

```
Título
Condominio
Fecha
Responsable
Resumen ejecutivo
Detalle de lo realizado
Observaciones
Recomendaciones
Evidencias fotográficas
Estado final
Próximos pasos
```

8.8 Módulo de comunicaciones

Permite enviar información al comité y a vecinos.

Canales:

- Publicación en portal/app.
- Push.
- Correo.
- Telegram.
- WhatsApp en fase posterior.

Tipos:

- Comunicado general.
- Comunicado urgente.
- Informe publicado.
- Aviso de mantención.
- Aviso de corte de servicio.
- Resumen mensual.

Reglas sugeridas:

Evento	Comité	Vecinos
Falla crítica	Sí	Sí
Mantención programada	Sí	Sí
Tarea menor interna	Opcional	No
Informe mensual	Sí	Opcional
Reclamo individual	Sí, si corresponde	No

8.9 Módulo de comité

Panel especial para miembros del comité.

Vista principal:

Condominio
Incidencias abiertas
Tareas en proceso
Mantenciones realizadas
Próximas mantenciones
Comunicados publicados
Informes disponibles

Funciones:

- Ver estado general.
- Comentar incidencias.

- Descargar informes.
- Revisar historial.
- Ver compromisos pendientes.

8.10 Módulo de vecinos

Vista más simple y controlada.

Funciones:

- Ver comunicados.
- Ver novedades publicadas.
- Recibir notificaciones.
- Reportar incidencias, si se habilita.
- Consultar documentos públicos.

Importante:

El vecino no debe ver toda la gestión interna. Solo la información aprobada para comunidad.

8.11 Módulo de historial del condominio

Todo evento relevante queda registrado en una línea de tiempo.

Ejemplo:

```
28/06/2026 - Mantención preventiva de bombas
27/06/2026 - Reparación luminaria Torre B
26/06/2026 - Falla portón vehicular resuelta
25/06/2026 - Informe mensual publicado
```

Debe permitir buscar por:

- Fecha.
 - Categoría.
 - Responsable.
 - Condominio.
 - Palabra clave.
 - Estado.
-

8.12 Módulo de IA

Funciones principales:

Clasificación automática

Entrada:

"Hay agua saliendo por el shaft del piso 3"

Salida:

Categoría: Agua / filtración
Prioridad: Alta
Requiere aviso: Sí
Destinatarios sugeridos: administrador y supervisor

Redacción de informes

Entrada informal:

"Se revisaron bombas, bomba 1 ok, bomba 2 con ruido, no hay fuga. Recomiendo
mantención."

Salida formal:

Durante la inspección del sistema de bombas se verificó el funcionamiento general del equipo. La bomba N°1 se encuentra operativa, mientras que la bomba N°2 presenta un ruido leve. No se detectaron filtraciones visibles. Se recomienda coordinar mantención preventiva.

Redacción de comunicados

Versiones:

- Formal para correo.
- Breve para push.
- Directa para WhatsApp/Telegram.
- Técnica para comité.
- Simple para vecinos.

Resúmenes mensuales

La IA debe generar:

- Resumen ejecutivo.
- Incidencias relevantes.
- Mantenciones realizadas.
- Pendientes.
- Recomendaciones.

9. Flujo principal del sistema

9.1 Reporte desde supervisor

```
Supervisor abre app
↓
Selecciona condominio
↓
Crea inspección o incidencia
↓
Adjunta foto/audio/texto
↓
Komite clasifica
↓
Komite genera borrador
↓
Administrador revisa
↓
Se informa al comité o vecinos
```

9.2 Reporte desde conserje

```
Conserje reporta novedad
↓
Supervisor recibe alerta
↓
Supervisor valida
↓
Se crea tarea
↓
Se resuelve
↓
Se genera evidencia
```

9.3 Informe mensual

```
Fin de mes
↓
Komite recopila tareas, incidencias y mantenciones
↓
IA genera informe mensual
↓
Administrador revisa
↓
Se publica al comité
```

10. Integraciones de comunicación

10.1 Telegram para MVP

Recomendado para:

- Supervisores.
- Empleados internos.
- Comité piloto.

Ventajas:

- API gratuita.
- Fácil recepción de mensajes.
- Soporte de fotos, audios y documentos.
- Ideal para validar el flujo de IA.

10.2 WhatsApp Business API futuro

Recomendado para:

- Vecinos.
- Comunicaciones sensibles.
- Respuestas a mensajes entrantes.

Uso sugerido:

- Primero como canal de recepción.
- Luego como canal de respuesta controlada.
- Comunicados masivos solo cuando tenga sentido.

10.3 Correo electrónico

Uso recomendado:

- Informes formales.
- Comunicados extensos.

- Documentos adjuntos.
 - Resúmenes mensuales.
-

10.4 Push móvil

Uso recomendado:

- Avisos rápidos.
 - Alertas internas.
 - Notificación al comité.
 - Recordatorios.
-

11. Web operativa tenant y backoffice Vallestar

Komite debe separar dos mundos distintos.

11.1 Web operativa Nuxt para empleados del tenant

Esta es la herramienta diaria de la empresa administradora que compra Komite.

Pantallas principales:

1. Dashboard empresa.
2. Dashboard por condominio.
3. Condominios.
4. Incidencias.
5. Tareas.
6. Inspecciones.
7. Informes.
8. Comunicaciones.
9. Usuarios del tenant.
10. Comité y vecinos.
11. Calendario de mantenciones.
12. Indicadores.
13. Configuración operativa del tenant.

Regla:

El empleado de una empresa administradora solo ve información de su propio tenant.

11.2 Backoffice SaaS Vallestar

Esta es la herramienta interna de Vallestar como proveedor del producto.

Pantallas principales:

1. Dashboard SaaS global.
2. Tenants / empresas administradoras.
3. Planes contratados.
4. Límites de uso.
5. Módulos habilitados.
6. Configuración global de IA.
7. Configuración de integraciones.
8. Auditoría global.
9. Logs técnicos.
10. Soporte.
11. Parámetros generales del producto.

Regla:

El backoffice de Vallestelar no se usa para operar condominios día a día.
Se usa para administrar la plataforma Komite como SaaS.

12. App móvil/tablet

Pantallas para supervisores

1. Inicio con tareas del día.
2. Selección de condominio.
3. Crear incidencia.
4. Crear inspección.
5. Capturar foto.
6. Grabar audio.
7. Checklist.
8. Tareas asignadas.
9. Historial del condominio.
10. Enviar informe a revisión.

Pantallas para comité

1. Dashboard del condominio.
2. Incidencias abiertas.
3. Tareas relevantes.
4. Informes.
5. Comunicados.
6. Historial.
7. Comentarios o solicitudes.

Pantallas para vecinos

1. Comunicados.

2. Novedades.
3. Alertas.
4. Reportar incidencia.
5. Documentos públicos.
6. Perfil.

13. Modelo de datos inicial

Tablas sugeridas:

```
tenants
tenant_plans
tenant_modules
tenant_settings
tenant_usage
condominiums
buildings
units
users
roles
user_tenants
user_condominiums
committee_members
residents
incidents
incident_events
tasks
task_checklists
inspections
inspection_templates
inspection_answers
reports
report_versions
communications
communication_recipients
attachments
audit_logs
ai_requests
notification_logs
integration_accounts
```

Regla técnica:

Las tablas operativas deben incluir tenant_id o estar relacionadas de forma inequívoca con una tabla que lo tenga.

14. Estados y trazabilidad

Cada acción importante debe guardar:

- Usuario.
- Fecha.
- Condominio.
- Acción realizada.
- Estado anterior.
- Estado nuevo.
- Evidencia adjunta.

Ejemplo:

```
28/06/2026 12:30
Supervisor Juan cerró la tarea "Revisión de bombas".
Adjuntó 4 fotografías.
IA generó informe preliminar.
Administrador aprobó a las 13:10.
```

15. Seguridad y permisos

Requisitos:

- Login con JWT.
- Roles por condominio.
- Separación estricta de datos entre tenants.
- Separación de permisos por condominio dentro de cada tenant.
- Auditoría de acciones.
- Control de acceso a documentos.
- Aprobación humana antes de comunicaciones masivas.

Reglas clave:

```
Un usuario de tenant solo puede ver información de su empresa administradora.
Dentro del tenant, solo puede ver los condominios donde está asignado.
Vallestelar puede administrar tenants, pero el acceso a datos operativos debe
estar auditado y limitado por soporte.
```

Recomendación técnica:

- Incluir `tenant_id` en JWT.
- Validar `tenant_id` en cada endpoint.
- Aplicar filtros automáticos por tenant en repositorios/servicios.
- Evitar endpoints globales sin control explícito.
- Registrar en auditoría cualquier acceso de soporte Vallestelar a datos de un tenant.

16. MVP recomendado

Para partir, no construiría todo.

El MVP debería incluir:

1. Gestión multi-tenant básica para Vallestar.
2. Gestión de condominios dentro de cada tenant.
3. Usuarios y roles básicos por tenant.
4. Web operativa Nuxt para empleados de la empresa compradora.
5. App móvil/tablet MAUI para supervisores y empleados de terreno.
6. Registro de incidencias con foto/audio/texto.
7. Tareas con estados.
8. Generación de informe con IA.
9. Aprobación manual del administrador del tenant.
10. Panel básico para comité.
11. Publicación de comunicados en portal/app.
12. Envío por correo.
13. Integración Telegram para uso interno.

Dejaría para fase 2:

- WhatsApp Business API.
- Notificaciones push avanzadas.
- App completa para vecinos.
- Firma digital.
- Analítica avanzada.
- Integración con Edifito.

17. Fases del proyecto

Fase 1 - MVP operativo

Objetivo:

Validar que supervisores puedan alimentar el sistema y que la IA genere informes útiles.

Incluye:

- Backoffice SaaS Vallestar para gestión de tenants.
 - Web operativa Nuxt para empleados del tenant.
 - App supervisor en .NET MAUI.
 - Incidencias.
 - Tareas.
 - Informes IA.
 - Comité con panel básico.
-

Fase 2 - Comunicación y vecinos

Objetivo:

Convertir Komite en canal de información para la comunidad.

Incluye:

- App/portal vecinos.
 - Comunicados.
 - Push.
 - Correo.
 - Segmentación por torre/unidad.
-

Fase 3 - Automatización avanzada

Objetivo:

Reducir trabajo administrativo repetitivo.

Incluye:

- WhatsApp Business API.
 - Telegram bot completo.
 - Informes mensuales automáticos.
 - Reglas de comunicación.
 - Respuestas sugeridas.
-

Fase 4 - Inteligencia operativa

Objetivo:

Transformar datos históricos en decisiones.

Incluye:

- KPIs.
 - Predicción de problemas recurrentes.
 - Ranking de proveedores.
 - Cumplimiento de mantenciones.
 - Comparativa entre condominios.
-

18. Arquitectura técnica sugerida

```
komite/  
├─ api/  
│   └─ app/  
│       └─ modules/
```

```
├── auth/
├── tenants/
├── plans/
├── condominiums/
├── users/
├── roles/
├── incidents/
├── tasks/
├── inspections/
├── reports/
├── communications/
├── attachments/
├── integrations/
├── ai/
├── core/
│   ├── tenant_context.py
│   ├── security.py
│   └── permissions.py
├── static/
│   ├── css/
│   ├── js/
│   └── images/
├── templates/
│   ├── vallestelar_login.html
│   ├── vallestelar_dashboard.html
│   ├── tenants.html
│   ├── plans.html
│   ├── ai_settings.html
│   └── audit.html
├── main.py
├── migrations/
└── Dockerfile

web/
├── nuxt.config.ts
├── pages/
│   ├── login.vue
│   ├── dashboard.vue
│   ├── condominiums/
│   ├── incidents/
│   ├── tasks/
│   ├── inspections/
│   ├── reports/
│   ├── communications/
│   └── users/
├── components/
├── composables/
└── package.json

mobile/
├── Komite.Mobile/
│   ├── Views/
│   ├── ViewModels/
│   └── Models/
```

```
├── Services/
├── Platforms/
├── App.xaml
└── Komite.Mobile.sln

worker/
├── app/
│   ├── ai/
│   ├── notifications/
│   ├── telegram/
│   └── jobs/
└── Dockerfile

docker-compose.yml
deploy.sh
.env.example
README.md
```

Notas:

- **api/** contiene la API FastAPI y el backoffice SaaS simple de Vallestelar.
- **web/** contiene la web operativa Nuxt para empleados de las empresas administradoras compradoras.
- **mobile/** contiene la app móvil/tablet .NET MAUI.
- **worker/** procesa IA, informes, notificaciones e integraciones.
- El aislamiento multi-tenant debe implementarse desde el backend, no solo desde el frontend.

19. Contenedores sugeridos

Docker	Compose
└─ web	Nuxt operativo para empleados del tenant
└─ api	FastAPI + REST API + Backoffice SaaS Vallestelar HTML/JS
└─ postgres	PostgreSQL
└─ redis	Redis
└─ worker	Python Worker para IA / jobs / notificaciones
└─ nginx	Reverse proxy
└─ telegram-bot	Opcional, puede ser parte del worker

La app .NET MAUI no corre como contenedor en producción. Se compila y distribuye como aplicación móvil/tablet para Android/iOS.

20. Dominios sugeridos

```
komite.cl
app.komite.cl
```

```
api.komite.cl  
admin.komite.cl
```

Asignación recomendada:

```
app.komite.cl      → Web operativa Nuxt para empleados de tenants  
api.komite.cl      → API FastAPI / Swagger / endpoints móviles  
admin.komite.cl    → Backoffice SaaS Vallestelar servido desde FastAPI
```

Opción futura para tenants grandes:

```
empresa-a.komite.cl  
empresa-b.komite.cl
```

En ese caso, el subdominio puede ayudar a resolver el tenant, aunque la seguridad nunca debe depender solo del subdominio. El backend siempre debe validar `tenant_id`, usuario, rol y permisos.

21. Variables de entorno sugeridas

```
APP_NAME=Komite  
ENVIRONMENT=production  
  
POSTGRES_DB=komite_db  
POSTGRES_USER=postgres  
POSTGRES_PASSWORD=change_me  
POSTGRES_HOST=postgres  
POSTGRES_PORT=5432  
  
REDIS_URL=redis://redis:6379/0  
  
JWT_SECRET_KEY=change_me  
JWT_ALGORITHM=HS256  
ACCESS_TOKEN_EXPIRE_MINUTES=1440  
  
API_PUBLIC_BASE_URL=https://api.komite.cl/api/v1  
MOBILE_API_BASE_URL=https://api.komite.cl/api/v1  
Nuxt_PUBLIC_API_BASE=https://api.komite.cl/api/v1  
Nuxt_PUBLIC_APP_NAME=Komite  
DEFAULT_TENANT_RESOLUTION=subdomain_or_login  
  
AI_PROVIDER=openai_or_mistral  
AI_API_KEY=change_me  
AI_MODEL=change_me  
  
TELEGRAM_BOT_TOKEN=change_me
```

```
SMTP_HOST=change_me
SMTP_PORT=587
SMTP_USER=change_me
SMTP_PASSWORD=change_me
SMTP_FROM=no-reply@komite.cl
```

22. Reglas de IA

Para evitar errores, la IA debe trabajar bajo reglas estrictas.

Reglas:

1. No inventar hechos.
2. No agregar información que no esté en el reporte original.
3. Diferenciar entre hecho, observación y recomendación.
4. Generar siempre borradores revisables.
5. Guardar el texto original junto al texto generado.
6. Indicar nivel de confianza si la información es ambigua.
7. Pedir validación humana para comunicaciones sensibles.

23. Ejemplo completo de uso

Entrada del supervisor

Audio:

"Estamos en sala de bombas de Vista Sol. Revisamos bomba uno y bomba dos. Bomba uno normal. Bomba dos con ruido leve. No vimos fuga. Recomendando pedir mantención preventiva."

Komite genera incidencia/tarea

Categoría: Bombas
Prioridad: Media
Estado: En revisión
Responsable sugerido: Supervisor técnico
Requiere comunicación: Comité sí, vecinos no necesariamente

Komite genera informe

Informe de revisión de bombas

Condominio: Vista Sol
Fecha: 28/06/2026
Responsable: Supervisor asignado

Resumen:

Se realizó una revisión del sistema de bombas del condominio. La bomba N°1 se encuentra funcionando normalmente. La bomba N°2 presenta un ruido leve durante su operación.

Observaciones:

- No se detectaron filtraciones visibles.
- Se recomienda coordinar mantención preventiva.

Estado:

Revisión realizada con observación pendiente.

Komite propone mensaje al comité

Estimados miembros del comité:

Informamos que se realizó una revisión del sistema de bombas del condominio. La bomba N°1 se encuentra funcionando normalmente y la bomba N°2 presenta un ruido leve, por lo que se recomienda coordinar una mantención preventiva.

No se detectaron filtraciones visibles.

Saludos,
Administración.

23.1 Ejemplo multi-tenant

Tenant: Administración Los Valles SpA

- └─ Condominio Vista Sol
- └─ Condominio Jardín Norte
- └─ Condominio Parque Central

Tenant: Gestión Condominios Sur Ltda.

- └─ Condominio Costa Azul
- └─ Condominio Altos del Mar

Un administrador de Administración Los Valles SpA puede ver Vista Sol, Jardín Norte y Parque Central según sus permisos internos.

Ese mismo usuario no puede ver Costa Azul ni Altos del Mar, porque pertenecen a otro tenant.

Vallestelar, desde el backoffice SaaS, puede ver que existen ambos tenants, su plan, estado de cuenta, módulos activos y métricas globales, pero el acceso a datos operativos debe quedar restringido, justificado y auditado.

24. Diferenciadores de Komite

1. No compite con Edifito: complementa la operación diaria.
 2. Convierte reportes informales en informes profesionales.
 3. Da trazabilidad al comité.
 4. Reduce la dependencia de WhatsApp desordenado.
 5. Permite evidencias fotográficas y audios.
 6. Crea historial consultable por condominio.
 7. Genera reportes mensuales automáticamente.
 8. Permite medir desempeño de la administración.
 9. Centraliza tareas, informes y comunicaciones.
 10. Da sensación de transparencia y control.
-

25. Indicadores clave

KPIs iniciales:

- Incidencias abiertas.
 - Incidencias cerradas.
 - Tiempo promedio de resolución.
 - Tareas vencidas.
 - Tareas por supervisor.
 - Mantenciones realizadas.
 - Comunicaciones enviadas.
 - Informes aprobados.
 - Incidencias por categoría.
 - Condominios con más pendientes.
-

26. Riesgos del proyecto

Riesgo 1: Sobrecargar a empleados

Solución:

- Formularios muy simples.
- Uso de audio y foto.
- Mínimos campos obligatorios.

Riesgo 2: Que la IA invente información

Solución:

- Plantillas cerradas.
- Aprobación humana.
- Texto original siempre visible.

Riesgo 3: Baja adopción de vecinos

Solución:

- Primero comité y supervisores.
- Luego vecinos con portal simple.
- WhatsApp solo en fase posterior.

Riesgo 4: Demasiado alcance inicial

Solución:

- MVP enfocado en supervisores, informes y comité.

27. Recomendación final

El mejor punto de partida para Komite es:

Multi-tenant SaaS + Web operativa Nuxt + App supervisor MAUI + Panel comité + IA de informes

No empezaría por vecinos ni por WhatsApp.

Primero validaría que la administradora pueda demostrar al comité:

- Qué se hizo.
- Cuándo se hizo.
- Quién lo hizo.
- Qué evidencia existe.
- Qué queda pendiente.

Ese es el corazón del producto.

Cuando ese flujo funcione, se añade comunicación masiva, app vecinos y WhatsApp Business.